

Part 2: Container Security

Implementation (30 points)

Docker Security Best Practices (10 points):

1. List and explain five Docker security best practices.

Below are the five docker security best practices:

- I. Trusted and Authorized based images: Use the trusted or authorized based image from docker hub or if you have any private harbor repository hosted in your private network.
- II. Run as Non-Root User: Don't run processes inside containers as root. Instead, define a user in your Dockerfile (USER appuser)
- III. Keep Secrets out of Images: Never hardcode API keys, passwords, or certificates in Dockerfile or commit them to Git instead use environment variables, Docker secrets, or external secret managers (like AWS Secrets Manager, Vault).
- IV. Regularly Scan Images for Vulnerabilities: Use tools like Trivy, Clair, or Docker Hub's built-in scanning to Detects CVEs in base images or libraries so you can patch/update early.
- V. Network Security & Isolation: Use user-defined networks, avoid exposing unnecessary ports, and don't run containers in --privileged mode. This limited the exposure of unnecessary ports to outside world and reduces the risk.

2. Implement one of these practices in a Dockerfile and provide the code.

I have saved a docker file with has mentioned docker security best practices along with the comment also attaching the snapshots for reference.

Docker file path: `tek_challenge/task-2/docker/`

- I. **For Regularly Scan Images for Vulnerabilities:** I'm using trivy. Below we do not find any vulnerabilities as the base image that we are using is latest and also it is and trusted, authorized image from docker hub. PFA snapshot

```

$ trivy image myapp:secure
2025-08-19T16:55:48.279+0530 INFO Need to update DB
2025-08-19T16:55:48.279+0530 INFO Downloading DB...
30.57 MiB / 30.57 MiB [-----] 100.00% 558.12 KiB p/s 56s
2025-08-19T16:56:52.206+0530 INFO Detected OS: debian
2025-08-19T16:56:52.206+0530 WARN This OS version is not on the EOL list: debian 13
2025-08-19T16:56:52.206+0530 INFO Detecting Debian vulnerabilities...
2025-08-19T16:56:52.206+0530 INFO Number of PL dependency files: 0
2025-08-19T16:56:52.206+0530 WARN This OS version is no longer supported by the distribution: debian 13.0
2025-08-19T16:56:52.206+0530 WARN The vulnerability detection may be insufficient because security updates are not provided

myapp:secure (debian 13.0)
=====
Total: 0 (UNKNOWN: 0, LOW: 0, MEDIUM: 0, HIGH: 0, CRITICAL: 0)

```

- II. **For network Security & Isolation:** For network security and isolation we can create a separate bridge network and using use below command to run container on custom bridge network and port
Command: `docker run -d --network my_secure_net --name secure_app -p 8080:8080 myapp:secure`
- III. **Run as Non-Root User:** Running the contain as non root user i.e., appuser. PFA snapshot.

```

$ docker run --rm myapp:secure id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)
$

```

Kubernetes Security Configuration (10 points):

1. Describe three Kubernetes security features.

- I. **Role-Based Access Control (RBAC):** RBAC controls /manges who can access the kubernetes API and provides the permissions to perform actions. It prevents unauthorized access from users or services to perform actions.
- II. **Network Policies:** It controls the connection or traffice between pods/services by defining which pod can communicate with other pod. Without policies, all pods can communicate with each other.
- III. **Pod Security Standards / Admission Control:** It defines security requirements for pods (e.g., disallow running as root, enforce read-only root filesystem). It is responsible to stop risky workloads from being scheduled (e.g., privileged containers, host network access).

2. Write a Kubernetes YAML configuration that includes securityContext settings for a pod.

YAML configuration file is present in the below path:

`tek_challenge/task-2/kubernetes/`

I. Pod Security Standards / Admission Control:

```
$ kubectl get pod secure-pod -o yaml | grep -A10 securityContext
{"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"labels":{"app":"myapp"},"name":"secure-pod","namespace":"default"},"spec":{"containers":[{"image":"myapp:secure","name":"myapp-secure","port":8080}],"resources":{"limits":{"cpu":"500m","memory":"256Mi"},"requests":{"cpu":"500m","memory":"256Mi"}}},"securityContext":{"allowPrivilegeEscalation":false,"capabilities":{"drop":["ALL"],"readOnlyRootFilesystem":true},"fsGroup":2000,"runAsNonRoot":true,"runAsUser":1000},"serviceAccountName":"default","terminationGracePeriodSeconds":30,"tolerations":[{"key":"node.kubernetes.io/not-ready","operator":"Exists","effect":"NoExecute","durationSeconds":30}]}
generation: 1
labels:
  app: myapp
name: secure-pod
namespace: default
resourceVersion: "927"
uid: 18634d01-8e22-4ad0-b1c6-7a2e1a56fafa
spec:
  containers:
    - securityContext:
        allowPrivilegeEscalation: false
        capabilities:
          drop:
            - ALL
        readOnlyRootFilesystem: true
        terminationMessagePath: /dev/termination-log
        terminationMessagePolicy: File
        volumeMounts:
          - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
            name: kube-api-access-5kqj
    - securityContext:
        fsGroup: 2000
        runAsNonRoot: true
        runAsUser: 1000
  serviceAccount: default
  serviceAccountName: default
  terminationGracePeriodSeconds: 30
  tolerations:
    - effect: NoExecute
      key: node.kubernetes.io/not-ready
      operator: Exists
```

II. Container user (non-root user i.e., appuser):

```
$ kubectl exec -it secure-pod -- id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser),2000
```

III. Container port is exposed to 8080 only

```
$ kubectl describe pod secure-pod | grep Port
Port: 8080/TCP
```

Summary:

The image is built and pushed from a trusted registry.

- The pod is running with securityContext applied.
- The container runs as non-root user (appuser).
- Only required port (8080) is exposed.

IaaS Security Measures (10 points):

1. Explain the concept of Infrastructure as a Service (IaaS) and its security implications.

Infrastructure as a Service (IaaS) means renting resources such as compute (VMs), storage, and networking over the internet on a pay-as-you-go basis. Where Provider manages the physical data centers, hardware and virtualization of the resources and customer controls the configurations, operating system, resources like CPU, RAM, Storage, Data, applications and many other security settings mentioned below:

- **Identity and access control:** Enforce least-privilege access using roles and groups, avoid long-lived admin keys. Require MFA for all privileged users. Rotate credentials and store secrets in a managed vault. Also review for the access managed at every preferred frequency.

- **Network security:** allow only required ports and IPs and deny all other endpoints. Segment networks (e.g., tiered subnets: web/app/db). Use cloud firewalls/security groups and,IDS/IPS or threat detection services for better security. Introduce WAF in front of public facing resources/applications.
- **OS hardening and patching:** Keep OS up to date. Implement automated scanning and applying patches after proper evaluation and testing.
- **Data protection:** Encrypt data at rest and in transit using custom managed key instead of cloud managed keys. Restrict public exposure to the resources which have critical data.
- **Backup, resilience, and DR:** Back up critical data and configurations (which includes VM snapshots). Test the restores and failover timely based on RTO and RPO. Use multiple regions and zones for HA.